Checkpointing and Resume Mechanism for CoMOLA Optimization Workflows

Objective

To enable continuation of long-running multi-objective optimization experiments by saving intermediate results and resuming execution from the last completed generation, thereby avoiding loss of computational progress.

Approach

A checkpointing mechanism was implemented to:

- Save the **Pareto front (population)** after each generation
- Store the **last completed generation number**
- Allow controlled resumption of the optimization process

A configuration parameter was introduced:
start_from_previous_gen = True / False

Execution Logic

- **If `start_from_previous_gen = False`:**
  - The algorithm starts from **generation 1**
  - Any previously saved data is ignored
- **If `start_from_previous_gen = True`:**
  - The system checks for an existing saved Pareto front
  - If not found:
    - A warning is issued and the run starts from scratch
  - If found:
    - The saved Pareto front is loaded
    - The last completed generation is identified
    - Optimization resumes from the **next generation**

Key Concept

- Optimization state is defined as:
  (Pareto front + generation number)
- Resuming the algorithm means continuing evolution from the last saved state rather than restarting the process.

Use Cases

1. Extending an Existing Run

- Initial run:

  max_gen = 10

- To extend:

  start_from_previous_gen = True
  max_gen = 20

An experiment is initially executed for 10 generations. After evaluating the results, it is determined that more generations are required. By enabling start_from_previous_gen and increasing the maximum number of generations to 20, the algorithm resumes from generation 11 and continues until generation 20.

Outcome:
The algorithm resumes from **generation 11** and continues until generation 20.

---

2. Recovering from Interrupted Execution

- Planned run:

  max_gen = 200

- Interrupted at generation 70
- Restart with:

  start_from_previous_gen = True

- max_gen = 200

An experiment is planned for 200 generations but is interrupted at generation 70 due to time or system constraints. Upon restarting the process with start_from_previous_gen enabled, the algorithm loads the saved state and resumes from generation 71, avoiding recomputation of the completed generations.

Outcome:
The algorithm resumes from **generation 71**, avoiding recomputation of previous generations.